Transformer Architecture

Hobhav S Fulzele

October 28, 2026

Contents

1 Introduction **2**

2 Tokenization **2**

3 Token Embeddings **3**

4 Attention Mechanism **4**

4.1 Intuition Behind Attention 4

4.2 Queries, Keys, and Values 5

4.3 Attention Score Calculation 6

4.4 Scaled Dot-Product Attention 7

4.5 Attention Masks 8

4.6 Self-Attention and Cross-Attention 8

5 Positional Information **9**

5.1 Sinusoidal Positional Encoding 9

6 Components of a Transformer Block **11**

6.1 Multi-Head Self-Attention 11

6.2 Position-Wise Feed-Forward Network 12

7 Pretraining, Fine-Tuning, and Inference **13**

7.1 Pretraining 13

7.2 Other Pretraining Objectives 14

7.3 Fine-Tuning 14

7.4 Pretraining Versus Fine-Tuning 15

1 Introduction

A Transformer is a deep learning architecture that processes sequences using attention mechanisms rather than relying primarily on recurrent computations. The original architecture was designed for neural machine translation and consisted of an encoder and a decoder, each built from attention and feed-forward layers.

Before Transformers, sequence-processing tasks commonly relied on recurrent neural networks (RNNs), including Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs). These models process a sequence one element at a time. This creates dependencies between consecutive computation steps and can make training difficult to parallelize.

Transformers approach the problem differently. They allow a token to interact directly with other tokens through attention. As a result, a token's representation can incorporate information from distant parts of a sequence without requiring that information to pass through every intermediate token.

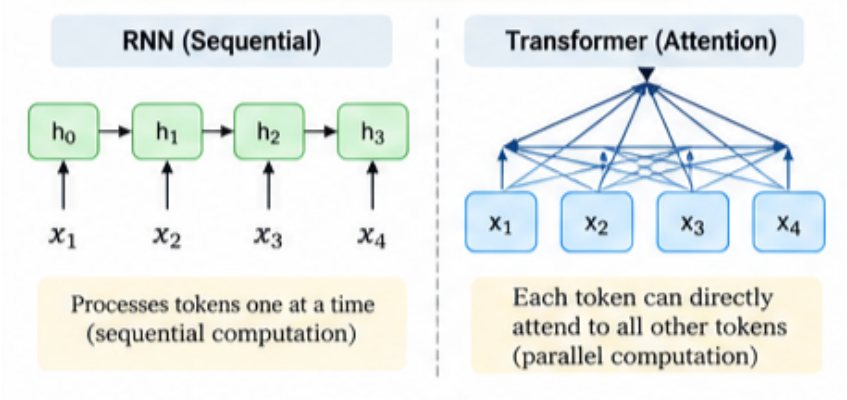


Figure 1: Comparison of sequential processing in RNN and parallel processing in Transformer

2 Tokenization

Tokenization is the process of dividing input text into smaller units called *tokens*. Depending on the tokenizer, a token may represent a complete word, a subword, punctuation, a character, or another text unit.

For example, the sentence: `Transformers are powerful.`
might be split into word-level tokens: `[Transformers,are,powerful,.]`
A subword tokenizer might instead split one or more words into smaller pieces. The exact result depends on the tokenizer and its vocabulary.

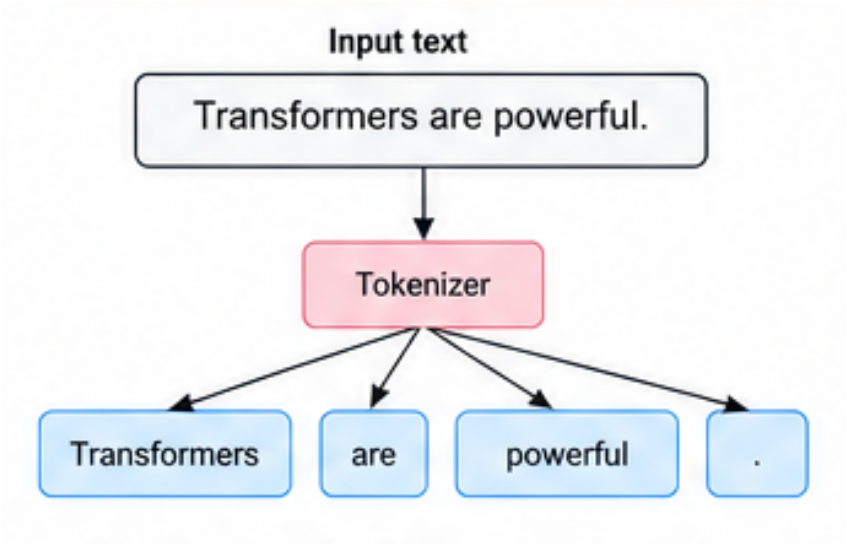


Figure 2: Example of tokenization, showing how an input sentence is split into tokens

3 Token Embeddings

An embedding converts each token ID into a dense vector of real-valued numbers. Suppose the vocabulary contains V tokens and the embedding dimension is d_{model}. The embedding matrix is

$$\mathbf{E} \in \mathbb{R}^{V \times d_{\text{model}}}.$$

Each row of $\mathbf{E}$ contains the embedding associated with one token. For token ID i, its embedding is

$$\mathbf{e}_i = \mathbf{E}[i, :].$$

For illustration, a token might have the four-dimensional embedding

$$\mathbf{e}_{\text{cat}} = \begin{bmatrix} 0.21 \\ -0.48 \\ 0.73 \\ 0.15 \end{bmatrix}.$$

Real Transformers typically use much larger embedding dimensions.

During training, embedding vectors are adjusted so that the model can use them effectively for its learning objective. Individual dimensions do not generally correspond to explicit human-defined properties. Instead, information is distributed across dimensions and represented through learned patterns and relationships.

A token embedding is not necessarily the final representation of a word. The same token can acquire different contextual representations after passing through Transformer layers. For example, *bank* has different meanings in "She deposited money at the bank" and "They sat on the river bank." Contextual processing helps distinguish these meanings.

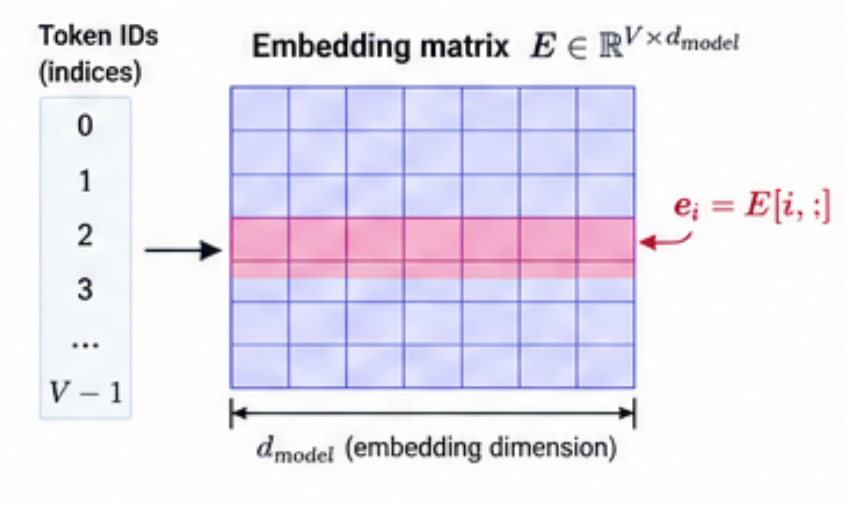


Figure 3: Token embedding matrix. Each row contains embedding vector of each token.

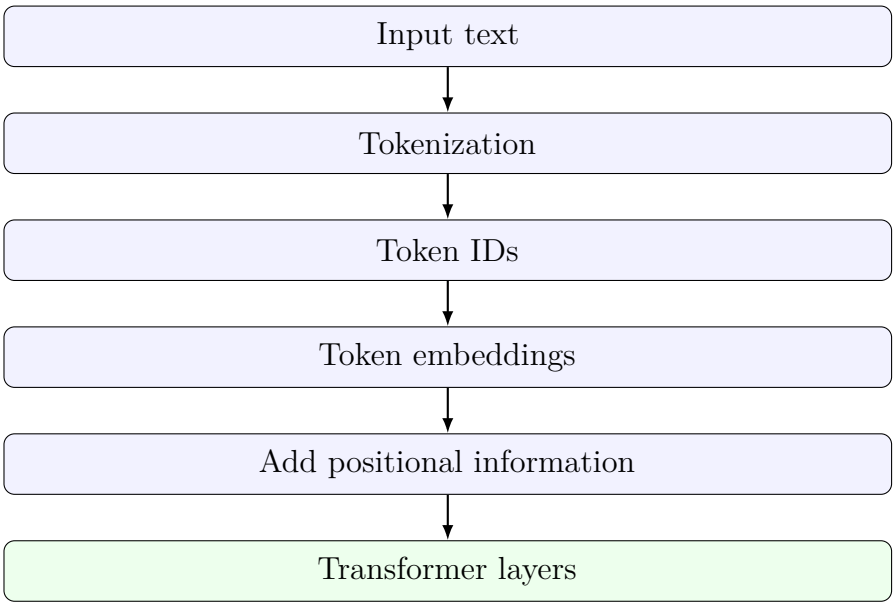


Figure 4: A simplified Transformer input pipeline.

4 Attention Mechanism

Attention is the central operation that allows a Transformer to determine which parts of a sequence are relevant when constructing a representation of a particular token. Rather than representing every token using only its own initial embedding, attention allows each token to gather information from other tokens.

4.1 Intuition Behind Attention

Consider the sentence: `The animal did not cross the street because it was tired.` To interpret *it*, a model may need to use information about *animal*. In another sentence,

the relevant reference might be a different noun. Attention gives the model a learned mechanism for assigning different weights to different tokens.

For a particular token, attention:

1. creates a query representing what information it is looking for;
2. creates keys representing information available from each token;
3. compares the query with the keys;
4. converts the scores into normalized weights; and
5. uses those weights to combine value vectors.

These vectors are called **Query (Q), Key (K), and Value (V)**.

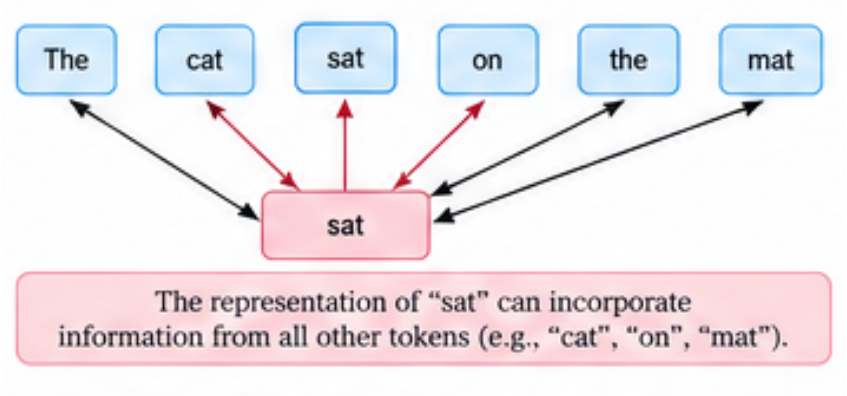


Figure 5: Intuition behind attention.

4.2 Queries, Keys, and Values

Let the input sequence representation be

$$\mathbf{X} \in \mathbb{R}^{L \times d_{\text{model}}}.$$

Three learned projection matrices transform the input into queries, keys, and values:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \tag{1}$$
$$\mathbf{K} = \mathbf{X}\mathbf{W}_K, \tag{2}$$
$$\mathbf{V} = \mathbf{X}\mathbf{W}_V. \tag{3}$$

Here, $\mathbf{Q}$ contains query vectors, $\mathbf{K}$ contains key vectors, and $\mathbf{V}$ contains value vectors. The matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ are learned parameters.

If the query and key dimensions are d_k, and the value dimension is d_v, then

$$\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{L \times d_k}, \qquad \mathbf{V} \in \mathbb{R}^{L \times d_v}.$$

An intuitive interpretation is:

- **Query:** What information am I looking for?
- **Key:** What information do I indicate that I contain?
- **Value:** What information do I contribute if selected?

These are conceptual analogies rather than literal questions stored in the network.

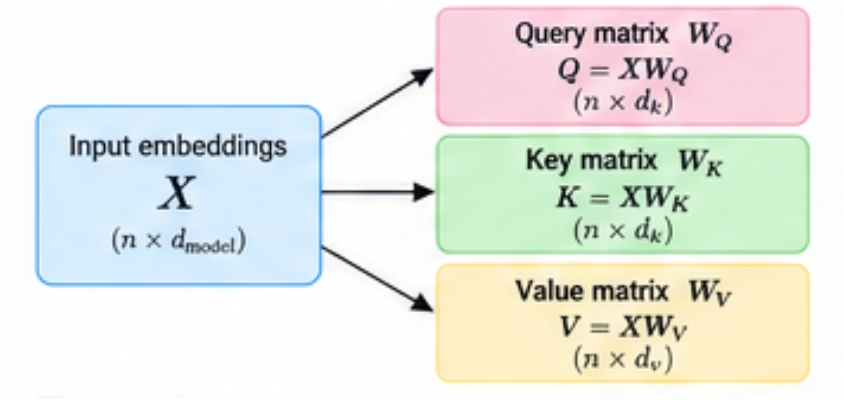


Figure 6: Comparing Key, Value, and Query from input embedding.

4.3 Attention Score Calculation

Attention begins by comparing each query with every key. The matrix of query-key similarities is

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T.$$

Each element is

$$S_{ij} = \mathbf{q}_i^T \mathbf{k}_j.$$

The score S_{ij} measures the compatibility between the query at position i and the key at position j. For a sequence of length L,

$$\mathbf{S} \in \mathbb{R}^{L \times L}.$$

Each row contains scores for one query token against all key tokens. These raw scores are not probabilities; they are scaled and normalized in the next steps.

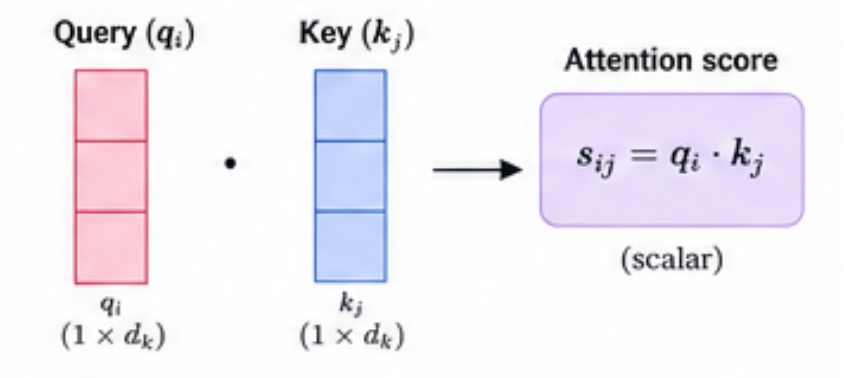


Figure 7: Attention score between token i and token j.

4.4 Scaled Dot-Product Attention

The original Transformer uses scaled dot-product attention:

$$\boxed{\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}}$$

where $\mathbf{M}$ is an optional attention mask. If no mask is required, this term can be omitted.

Here, d_k denotes the dimension of the key vectors and, correspondingly, the query vectors. The term $\sqrt{d_k}$ is the square root of this dimension and acts as a scaling factor for the dot-product attention scores.

4.4.1 Stage 1: Scale the Scores

$$\mathbf{S}_{\text{scaled}} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}.$$

When the key dimension becomes large, dot products can grow in magnitude. Large scores can make softmax highly concentrated, producing small gradients for many inputs. Scaling helps control score magnitude and supports stable optimization.

4.4.2 Stage 2: Apply Softmax

Softmax converts a row of scores into nonnegative weights that sum to one:

$$\alpha_{ij} = \frac{\exp(S_{ij}/\sqrt{d_k})}{\sum_{k=1}^{L} \exp(S_{ik}/\sqrt{d_k})},$$

where S_{ij} denotes the unscaled dot-product score. For a given query token i,

$$\sum_{j=1}^{L} \alpha_{ij} = 1.$$

The value α_{ij} determines how strongly that query incorporates information from key position j.

4.4.3 Stage 3: Compute the Weighted Sum

The output for token i is

$$\mathbf{z}_i = \sum_{j=1}^{L} \alpha_{ij}\mathbf{v}_j.$$

The output is a weighted combination of the value vectors. Consequently, the representation at one token position can incorporate information from other positions in the sequence.

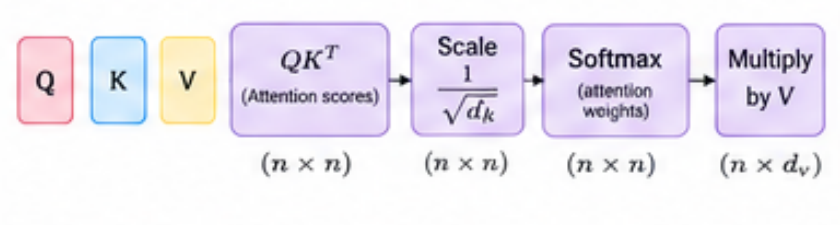


Figure 8: Scaled dot product attention.

4.5 Attention Masks

Masks restrict which tokens can be considered during attention. Two important types are:

- **Padding masks:** Prevent the model from treating padding positions as meaningful input tokens.
- **Causal masks:** Prevent a token from attending to future tokens. These are essential for standard autoregressive language models that predict the next token.

For a causal mask, the conceptual form is

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Adding negative infinity to a score causes its softmax weight to be zero. Thus, future positions receive no attention weight.

Standard encoder self-attention is typically bidirectional, whereas decoder self-attention in autoregressive generation is causal.

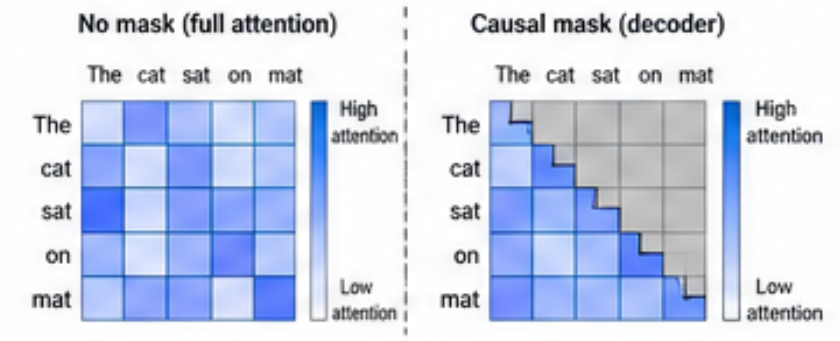


Figure 9: Mask Attention. **Left**: Full Attention. **Right**: Masked Attention

4.6 Self-Attention and Cross-Attention

In **self-attention**, queries, keys, and values are derived from the same sequence representation. In **cross-attention**, queries come from one representation while keys and values come from another. Cross-attention is particularly important in encoder–decoder Transformers, where the decoder attends to encoder outputs.

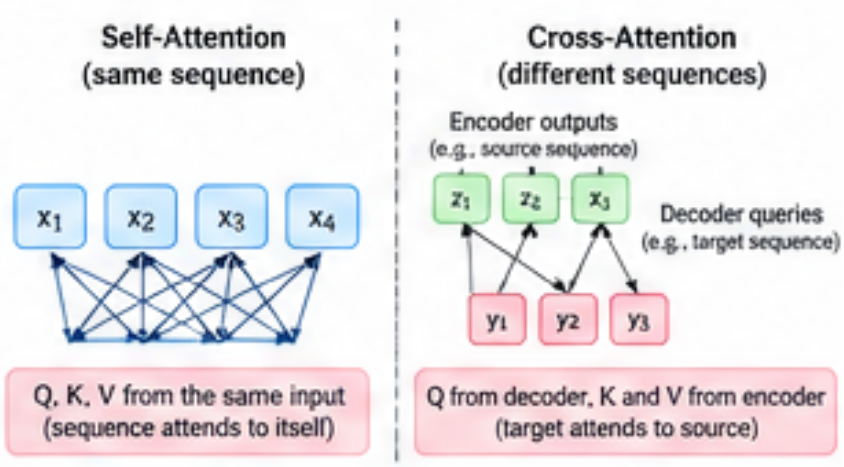


Figure 10: Self-Attention Vs Cross-Attention

5 Positional Information

Self-attention compares token representations, but the attention operation itself does not inherently encode the order in which tokens appeared. Consider:

- "The dog chased the cat."
- "The cat chased the dog."

These sentences contain the same words but have different meanings because the positions and relationships between words differ.

A Transformer therefore needs a way to represent token positions or positional relationships. In the original Transformer, positional encodings are added to token embeddings before they enter the encoder or decoder.

Let $\mathbf{E}$ denote token embeddings and $\mathbf{P}$ positional encodings. Then

$$\boxed{\mathbf{X} = \mathbf{E} + \mathbf{P}}$$

with

$$\mathbf{E}, \mathbf{P} \in \mathbb{R}^{L \times d_{\text{model}}}.$$

The addition combines information about token identity with information about position.

5.1 Sinusoidal Positional Encoding

The original Transformer uses sinusoidal functions to generate positional encodings. For position pos and dimension index i,

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

and

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

Even dimensions use sine functions and odd dimensions use cosine functions. Different dimensions have different wavelengths: some change rapidly across positions, while others vary more slowly. This creates a position-dependent vector for each token.

For illustration, a simplified four-dimensional positional vector could be

$$\mathbf{p}_{pos} = \begin{bmatrix} \sin(pos) \\ \cos(pos) \\ \sin(pos/100) \\ \cos(pos/100) \end{bmatrix}.$$

This is a simplified example of the frequency pattern, not the exact four-dimensional formula for every model configuration.

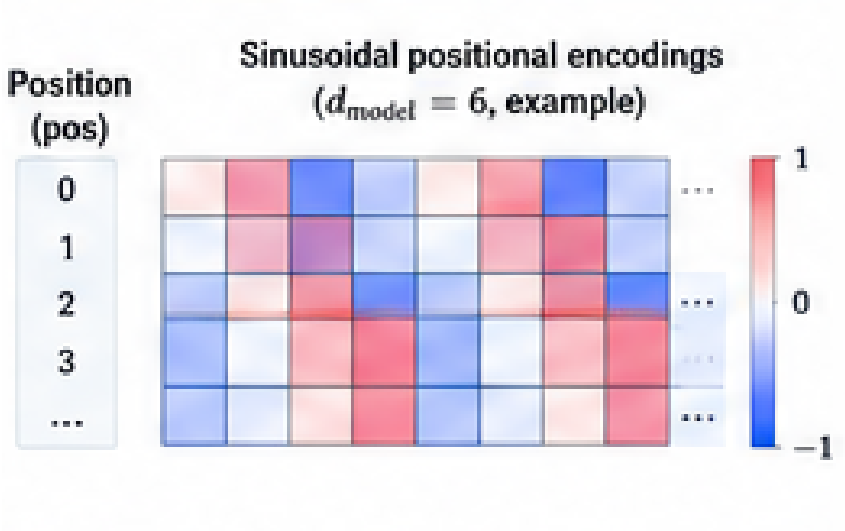


Figure 11: Sinusoidal Position encoding for different position.

Transformer Architecture

The original Transformer, introduced in *Attention Is All You Need* [**?**], has an encoder–decoder architecture. Its three main parts are:

1. **Encoder:** Processes the input sequence and produces contextual representations.
2. **Decoder:** Generates an output sequence using previously generated tokens and information from the encoder.
3. **Output projection and softmax:** Convert decoder representations into probability distributions over the vocabulary.

The original paper used six encoder layers and six decoder layers in its base architecture. Modern Transformers can use different depths and configurations.

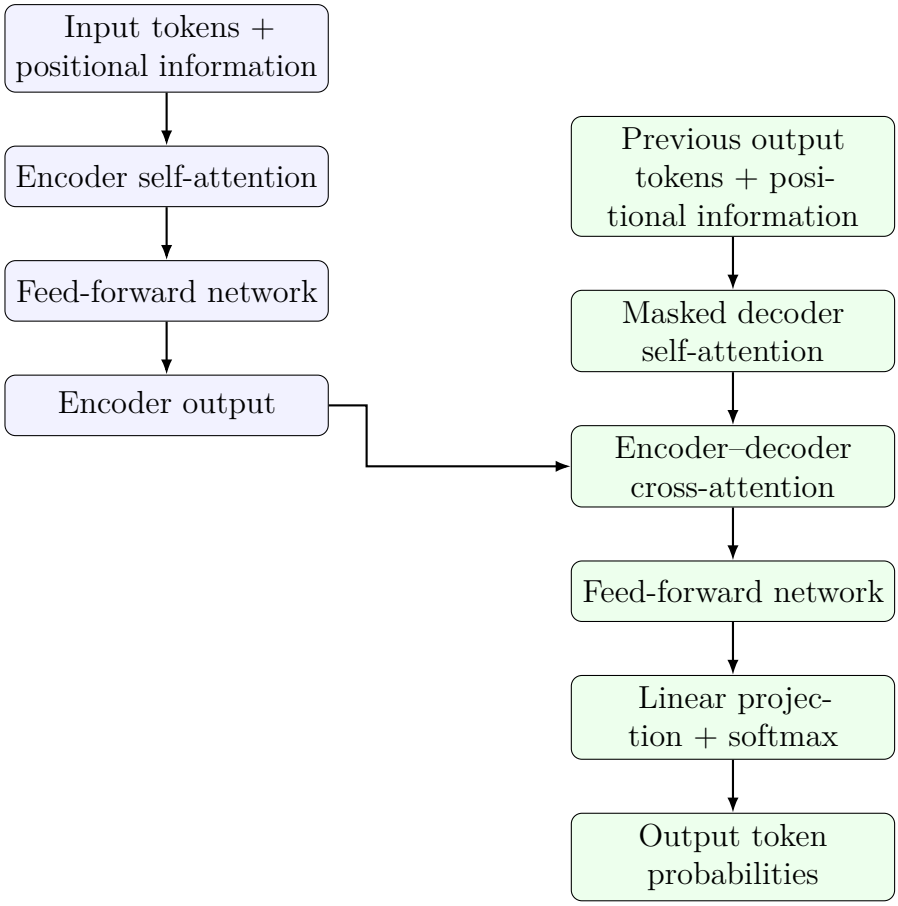


Figure 12: Simplified encoder–decoder Transformer architecture. In practice, the encoder and decoder contain stacks of repeated layers, and residual connections and normalization surround the main sublayers.

6 Components of a Transformer Block

6.1 Multi-Head Self-Attention

Multi-head attention extends the attention mechanism by computing several attention operations in parallel. The goal is to let the model represent different relationships between tokens rather than relying on one attention pattern. Different heads may learn to emphasize nearby tokens, long-distance dependencies, syntactic relationships, or information relevant to a contextual feature. These are possible learned behaviors, not fixed functions assigned to individual heads.

For head i,

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i).$$

The outputs are concatenated and transformed:

$$\boxed{\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O}$$

where h is the number of heads and $\mathbf{W}_O$ is the output projection matrix.

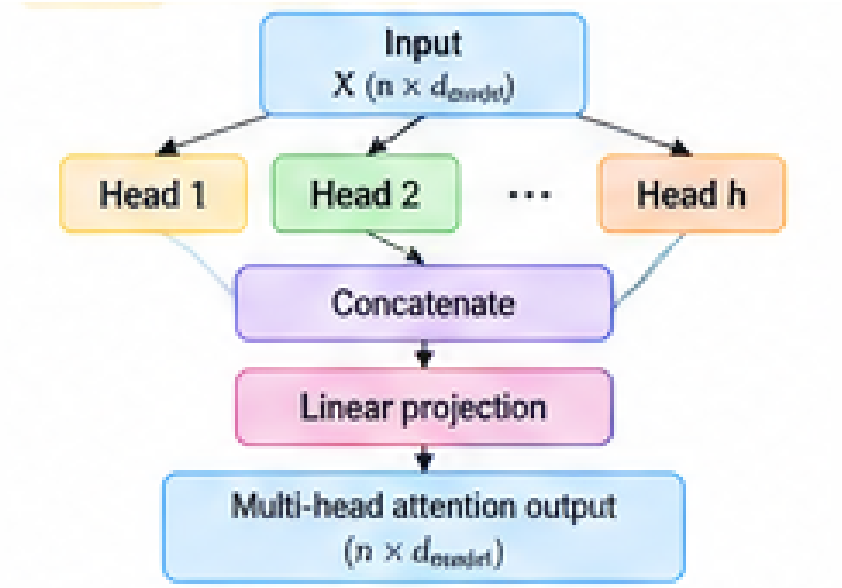


Figure 13: Multi-Head Self-Attention.

6.2 Position-Wise Feed-Forward Network

Attention mixes information across token positions. A feed-forward network then applies learned transformations to each position independently. In the original Transformer, the feed-forward sublayer consists of two linear transformations with a ReLU activation between them:

$$\boxed{\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2}$$

The first projection expands the hidden dimension, and the second projects it back to the model dimension.

The same parameters are applied at every sequence position.

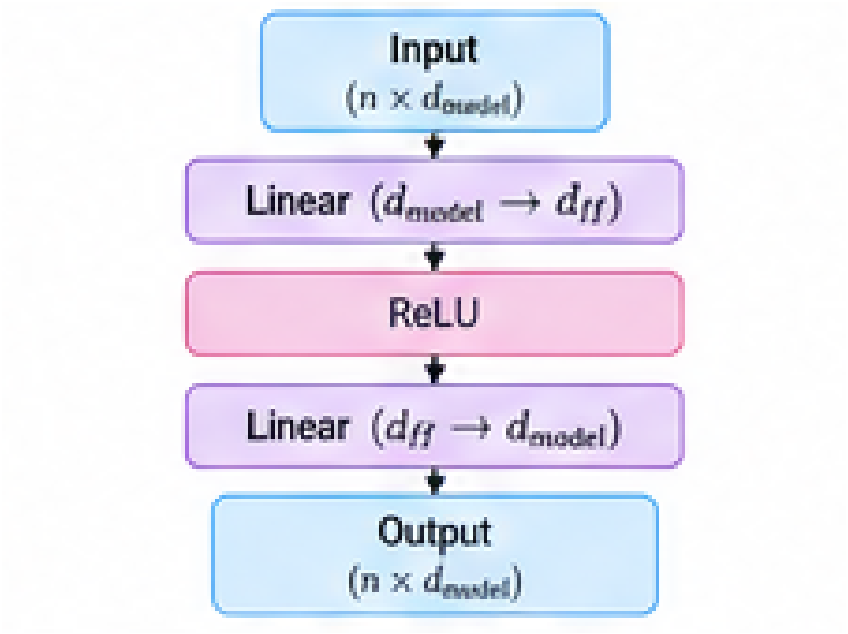


Figure 14: Position-Wise Feed-Forward Network.

7 Pretraining, Fine-Tuning, and Inference

The Transformer architecture describes how a model processes information. Pretraining and fine-tuning describe how its parameters are learned and adapted. Models with the same basic architecture can behave differently depending on their training data, objective, and optimization process.

7.1 Pretraining

Pretraining is the initial training stage in which a model learns general-purpose representations or predictive capabilities from a large dataset. For language models, this often involves large collections of text, although multimodal models can also use images, audio, video, and other data.

The model learns by optimizing an objective function that measures how well its predictions match the training targets.



Figure 15: Learn general language information from large amount of data.

7.2 Other Pretraining Objectives

Not every Transformer is pretrained using next-token prediction.

Objective	Description	Example
Causal language modeling	Predict the next token from preceding tokens	GPT-style models
Masked language modeling	Predict masked tokens using surrounding context	BERT
Sequence-to-sequence denoising	Reconstruct target text from corrupted input	T5
Contrastive learning	Align representations of related examples	CLIP-style models

Table 1: Examples of Transformer pretraining objectives.

7.3 Fine-Tuning

Fine-tuning takes a pretrained model and further trains it on a more specific dataset or task. The model begins with parameters learned during pretraining and updates them to improve performance on the target task.

Applications include sentiment classification, medical text classification, legal document extraction, customer support dialogue, domain-specific summarization, and instruction following.

If the pretrained model parameters are $\theta_{\text{pretrained}}$, fine-tuning produces adapted parameters:

$$\theta_{\text{fine-tuned}} = \text{Optimize}\left(\theta_{\text{pretrained}}, \mathcal{D}_{\text{target}}\right),$$

where $\mathcal{D}_{\text{target}}$ is the task-specific training dataset.

In full fine-tuning, most or all model parameters may be updated. In parameter-efficient fine-tuning, only a small subset of parameters or additional trainable components are updated.

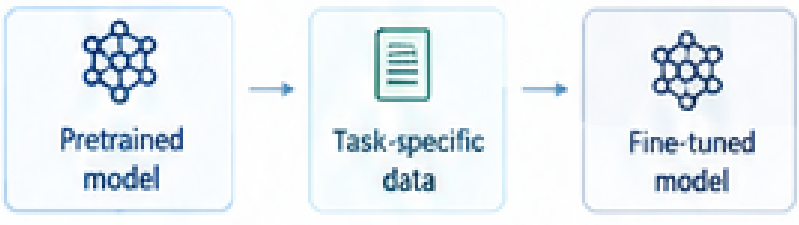


Figure 16: Adapt model to specific task.

7.4 Pretraining Versus Fine-Tuning

Aspect	**Pretraining**	**Fine-tuning**
Primary goal	Learn general capabilities and representations	Adapt an existing model to a task or desired behavior
Dataset	Usually broad and large	Often narrower and task-specific
Initialization	Initialized parameters	Pretrained parameters
Compute requirements	Often very high	Usually lower, depending on model and data
Objective	General language or multimodal objective	Task-specific, instructional, or preference-based objective
Outcome	General-purpose pretrained model	Adapted model

Table 2: Comparison of pretraining and fine-tuning.

Fine-tuning is not the same as inference. Fine-tuning updates model parameters, whereas inference uses trained parameters to produce predictions.